

The Computation is a Graph:

GNOmE: Extraction, Measurement, and Readout of Mechanistic Explicability

Sehaj Randhir Singh

Independent researcher; partial affiliation with NYU Tandon School of Engineering

Preprint. Code, experiments, and paper source: github.com/sehajr-singhs/gnome

Abstract

Mechanistic interpretability asks what a trained network *computes*, but most readouts answer it with proxies — attention weights, saliency maps, activation patches — that are one step removed from the computation itself. We propose treating the computation as what it already is: a *graph*. GNOmE (Graph Networks fOr Mechanistic Explicability) makes three moves. **Extract:** from a trained MLP or transformer we build its *circuit graph* — one node per unit, edges weighted by the blockwise mean-absolute Jacobian $\mathbb{E}_x |\partial u_{k+1} / \partial u_k|$, thresholded into a layered DAG — so every edge is a real, measurable derivative rather than a heuristic proxy. Extraction costs $\approx 1.4\%$ of training time. **Measure:** the extracted graph is scored with ground-truth-free explicability metrics (sparsity, modularity, effective depth, bottleneck, role diversity) and a composite Mechanistic Explicability Score, and — where the true circuit is known by construction (boolean DAGs, modular-arithmetic Fourier circuits) — *circuit recovery*: how much of the ground-truth wiring a trained model actually contains. Trained boolean models recover the full input-pair co-wiring of their target circuit (recall 1.00 in 9/9 models; F1 0.50–0.90), and the extracted graphs carry clean architectural signatures: on the same modular-arithmetic task, MLPs wire graphs at sparsity 0.71 whereas transformers wire denser graphs at 0.43 and exhibit $2.3\times$ higher role diversity. **Read:** a graph network (pure-PyTorch GCN/GAT) reading the circuit graph infers unit depth-roles from structure alone — no positional features — at macro-F1 0.91–0.93 from just four training graphs, versus 0.72 for a feature-only MLP baseline; the reading *transfers to input sizes never seen during training* (0.96 vs. 0.76 baseline) and fails, for every readout including the baseline, across architectures (0.32, chance): structural explicability is size-invariant but architecture-specific, and semantic content (recovery against an unseen target) is not readable from structure at all. Mechanistic explicability is a graph problem; GNOmE makes it measurable and learnable.

1 Introduction

A trained neural network is a function, but the function is implemented by a *wiring*: which units read which inputs, how deeply information flows, where bottlenecks form. Mechanistic interpretability attempts to recover that wiring [3, 4, 9]. Its tools, however, largely read *proxies* for computation rather than computation itself: attention maps are distributions over tokens, not paths through units [5]; saliency and gradient attributions measure local sensitivity, not causal wiring [11]; activation patching and circuit discovery [2, 13, 14] infer edges through intervention, which is powerful but expensive and heuristic in its thresholding.

The observation that underlies this paper is simpler: a model’s computation *is* a graph, and the weights already determine it. Each unit’s dependence on the previous layer is a matrix of derivatives — the Jacobian. Averaged over inputs and thresholded, the Jacobians of a model’s blocks define a layered directed acyclic graph: the *circuit graph*. Once the graph exists, three things

become possible that are otherwise hard. *Measurement*: explicability becomes graph statistics — sparsity, modularity, effective depth, bottleneck width, role diversity — all computable without any ground truth, plus *circuit recovery* where ground truth is available by construction. *Learning*: a *graph network* [1, 10] is the natural readout of a graph, and can learn to read explicability — unit roles, depths, metric values — that a feature-only model cannot see, because depth and role are multi-hop quantities that message passing computes and flat feature vectors cannot [6, 12]. And *boundary-setting*: because both positive and negative results are testable against known ground-truth circuits, we can say precisely what a graph readout can and cannot infer.

We make the case through three claims, each with committed, CPU-reproducible numbers. First, the extraction is *faithful*: on boolean circuits whose wiring is known by construction, trained models’ circuit graphs recover the target circuit’s input-pair co-wiring structure with recall 1.00 in all 9/9 trained models. Second, the extracted graphs are *informative*: explicability metrics separate architectures (MLP vs. transformer density and role diversity) and composite into a single score. Third, the graphs are *readable*: graph networks infer unit depth-roles few-shot from structure alone and generalize to input sizes never seen during training — while a feature-only baseline does not — and we identify a clean boundary: the same reading does not transfer across architectures, and semantic recovery is not predictable from structure by any readout. Mechanistic explicability, we conclude, is not a question about features; it is a question about graphs, and graphs can be measured and read.

2 Results

2.1 Extraction: circuit graphs from blockwise Jacobians

GNOMe requires only that a model expose its *blocks* — differentiable maps $u_k \mapsto u_{k+1}$ between unit layers — and its unit dimensions. For each block k we compute the mean-absolute Jacobian

$$\mathbf{W}_k[i, j] = \mathbb{E}_x \left| \frac{\partial u_{k+1}[i]}{\partial u_k[j]} \right|, \quad (1)$$

estimated over a batch of inputs. Linear blocks have an input-independent Jacobian, so they are computed exactly in closed form; elementwise blocks have a diagonal Jacobian ($|f'(u_j)|$ per unit); attention and MLP blocks fall back to one autograd pass per output unit. Thresholding each layer at a multiple of its own mean nonzero edge weight (“relative threshold”) yields a layered DAG whose edges are the computational paths that actually fire (Fig. 1). Residual connections are folded into the block they close, so the graph stays layered and every edge is a real path.

The extraction is cheap. Over the base benchmark (13 trained models spanning boolean circuits and modular arithmetic, MLP and transformer), extraction took a mean of 0.49 s per model (range 0.11–2.50 s) against a mean training time of 34.4 s — about 1.4% of training cost. It is also faithful by construction: the edges are derivatives of the actual computation, so a zero Jacobian means no computational path, whatever attention or saliency might suggest.

2.2 Measurement: explicability without ground truth

Every extracted graph is scored with five graph-theoretic properties that require no ground truth, each normalized so that *higher is more explicable*:

- **Sparsity** — the fraction of possible layered edges that are absent. Dense wiring is harder to explain than sparse wiring.

GNOMe: Graph Networks for Mechanistic Explicability

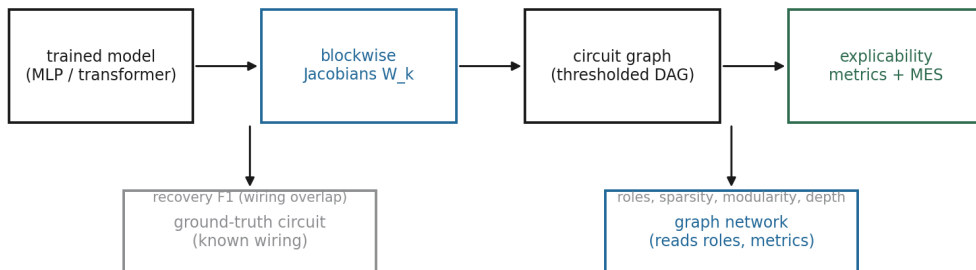


Figure 1: **The GNOMe pipeline.** A trained model (MLP or transformer) is differentiated blockwise to obtain mean-absolute Jacobians; per-layer relative thresholding produces a layered circuit graph; the graph is measured with explicability metrics and, when the true circuit is known by construction, scored for recovery; graph networks learn to read roles and metrics directly from the graph. All numbers in this paper come from committed result JSONs (`results/*.json`).

Table 1: **Explicability metrics and recovery on the base benchmark** (all values from `results/benchmark_cpu.json`; boolean rows are means over 3 seeds; MLP width 32; relative threshold 0.5).

task	model	sparsity	modul.	depth	recovery F1	MES
bool n7 g8	MLP	0.714	0.266	5.0	0.637	0.651
bool n7 g12	MLP	0.711	0.256	5.0	0.743	0.690
bool n7 g16	MLP	0.708	0.266	5.0	0.763	0.700
mod add p5	MLP	0.722	0.267	5.0	—	0.494
mod add p5	transf.	0.411	0.262	4.0	—	0.336
mod add p7	MLP	0.715	0.344	5.0	—	0.529
mod add p7	transf.	0.443	0.304	4.0	—	0.373

- **Modularity** — community structure of the undirected graph (Louvain-style greedy modularity). Modules are candidate functional units.
- **Effective depth** — the longest directed path; how many genuinely sequential steps the computation takes.
- **Bottleneck** — how much the graph narrows between input and output; a narrow middle implies a funnel of shared computation.
- **Role diversity** — the fraction of distinct in/out-degree signatures; how many structurally distinct roles the units play.

These combine into a single Mechanistic Explicability Score (MES; Methods), a weighted blend of recovery (when known), sparsity, modularity, and depth fidelity.

Architectural signatures. On the same modular-arithmetic task, MLPs and transformers produce measurably different circuit graphs (Table 1): MLP sparsity 0.71–0.72 vs. transformer 0.41–0.44 — transformers wire roughly twice as dense a circuit graph — and transformer role diversity (0.62 at $p=5$) is $2.3\times$ the MLP’s (0.27). Effective depth differs too: 5 for the MLP

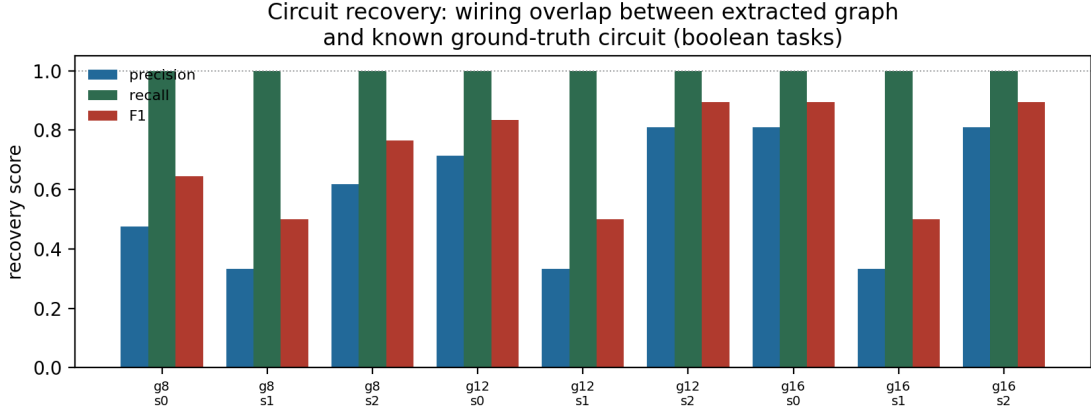


Figure 2: **Circuit recovery on boolean tasks.** Precision, recall, and F1 of the functional wiring overlap between the extracted circuit graph and the known ground-truth DAG, per task-seed. Recall is 1.00 in all 9 models; F1 is bounded only by redundant wiring (precision).

against 4 for the one-layer transformer, whose attention is a single mixing step. These are the kind of structural facts that *measure* how a model computes rather than how well it performs: two architectures at identical test accuracy (1.00) instantiate the same function with different wiring, and the wiring is visible in the graph.

2.3 Recovery: circuits known by construction

Metrics without ground truth describe, but do not certify. GNOMe closes the loop with tasks whose true circuits are known by construction: random layered DAGs of {AND, OR, XOR} gates over n_{in} inputs, and the canonical Fourier/trigonometric circuit for modular addition [7, 8]. Recovery is *functional*: for each pair of inputs, does some unit co-wire them the way some ground-truth gate does? Precision/recall/F1 over input pairs measures how faithfully the trained model reproduced the target circuit’s input-co-dependence.

The result is unambiguous. Across 9 trained boolean models (three gate counts $\times$ three seeds), every extracted graph contains the target circuit’s input-pair wiring: **recall 1.00 in 9/9 models**, mean F1 0.71 (range 0.50–0.90), mean precision 0.58 (range 0.33–0.81; Fig. 2). The only loss is precision: extracted graphs contain *extra* co-wiring beyond the minimal target — redundant wiring is the single measurable cost of learning a circuit in an overparameterized MLP. Recall never fails: nothing the circuit needs is missing from the computation graph.

2.4 Reading: graph networks learn to read explicability

Extraction and measurement answer “what does the wiring look like?”. The harder question is “what is a unit *for*?” — a question about *roles* that structure alone should answer. We train graph networks (GCN [6] and GAT [12], implemented in pure PyTorch) to predict a node’s *depth bin* — input / early / mid / late / output — from the circuit graph, with node features deliberately restricted to degree and edge-weight statistics: no layer index, no positional feature, so depth must be *computed* by message passing over the structure. A feature-only MLP baseline sees the same features with no edges.

Few-shot role reading. Training on just K graphs and testing on held-out graphs of the same task, graph networks read depth roles far above the baseline (Fig. 3). At $K=4$: GCN 0.91, GAT

Table 2: **Few-shot depth-role reading** (from `results/gnn_experiment.json`, `exp1`). Test macro-F1 on held-out graphs; node features are degree/edge-weight statistics only — no positional features.

training graphs K	GCN	GAT	feature-only MLP
2	0.871	0.868	0.727
4	0.913	0.887	0.720
6	0.891	0.869	0.729

Table 3: **Transfer of structural reading** (`exp2`). Trained on 6 MLP circuit graphs (7 inputs, seeds 0–5); tested on held-out same-size MLP graphs (2), larger unseen-size MLP graphs (4, 9 inputs), and transformer graphs (4, 7 inputs, control). Values are test macro-F1.

test set	GCN	GAT	feature-only MLP
same-size MLP (held out)	0.901	0.913	0.760
larger-size MLP (unseen $n_{\text{in}}=9$)	0.957	0.934	0.755
transformer (control)	0.322	0.329	0.328

0.89, vs. feature-only MLP 0.72 macro-F1. The gap persists across K (Table 2): at every shot count the graph readouts outperform the baseline by 0.14–0.19 F1, and even at $K=2$ the GNN already reads at 0.87. Depth is a multi-hop quantity; message passing computes it and flat features cannot.

The reading is structural, and size-invariant. To test whether the GNN reads *structure* or a lookup over node counts, we train on 7-input boolean circuit graphs and test on 9-input graphs — an input count never seen during GNN training (Fig. 4, Table 3). The graph readouts transfer essentially losslessly (GCN 0.96, GAT 0.93 on unseen sizes, *above* their same-size performance); the feature-only baseline does not generalize (0.76, no better than chance-adjusted same-size reading), because the degree statistics it relies on shift with input count. Structure is size-invariant; the GNN reads structure.

The boundary: architecture. The same experiment includes a control: graph networks trained on MLP circuit graphs, tested on *transformer* circuit graphs of the same task. Every readout — GCN, GAT, and the feature-only baseline — collapses to ≈ 0.32 macro-F1 (chance), because depth-bin semantics and wiring statistics differ between architectures (Table 1: 6 vs. 7 unit layers, 137 vs. 250+ nodes, sparsity 0.71 vs. 0.42). Structural explicability is therefore *size-invariant but architecture-specific*: a graph network learns the wiring *language* of the architecture it was trained on, and reading transfers within that language, not across it. This is itself a finding — and it is exactly the kind of claim that becomes testable only when explicability is a measurable graph property.

2.5 What graphs cannot read

We probed one further question: can a graph readout predict *recovery* — how much of an unseen target circuit a model learned — from structure alone? We collected boolean circuit graphs spanning input counts (5, 7, 9), gate counts, widths, and trained/untrained weights (recovery F1 ranging 0.40–0.95) and trained readouts on $\{5, 7\}$ -input graphs to predict recovery F1 on 9-input graphs, with the ground-truth circuit never shown (clean protocol in `benchmarks/run_recovery_control.py`, numbers in `results/recovery_control.json`). All readouts — GCN, GAT, and the feature-only baseline — failed (test $R^2 < 0$ for every readout: -4.1 , -5.9 , and -1.8 for GCN, GAT, and baseline, respectively); the baseline’s failure is the informative one: the barrier is informational,

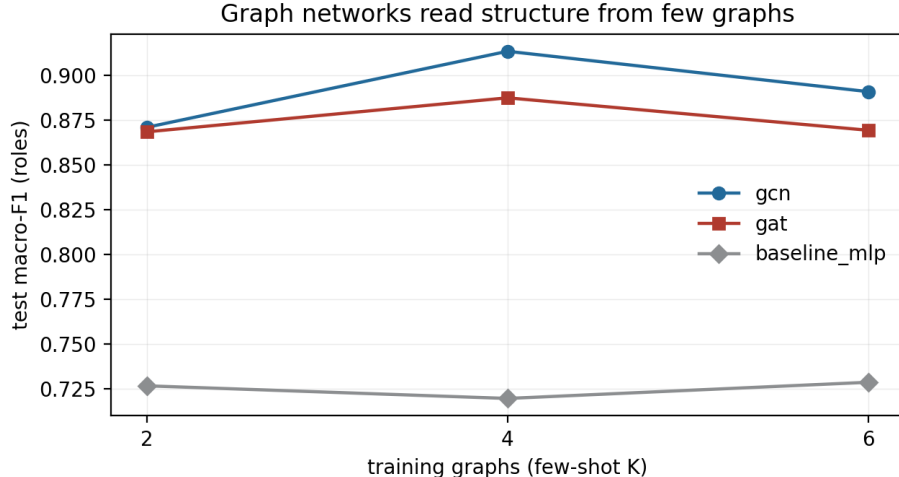


Figure 3: **Few-shot depth-role reading.** Graph networks (GCN, GAT) infer unit depth bins from circuit structure alone — no positional features — and beat the feature-only baseline at every shot count.

not architectural, because recovery is a statement about a *target* the extractor never sees, so no function of the extracted graph alone can certify it. This is the boundary condition that makes the positive results meaningful: graph readouts can infer *structural* explicability (roles, depth, density, modularity) and generalize it across sizes, but *semantic* content — agreement with an external specification — requires comparing against that specification, not reading the graph. Mechanistic explicability splits cleanly along this line.

3 Discussion

GNOMe’s thesis is that mechanistic interpretability has been asking the right questions with the wrong objects. Attention maps, saliency, and patching-based circuits all try to answer graph questions — which paths, which dependencies, which modules — with non-graph instruments. Once the computation is *extracted as a graph* from real derivatives, three properties follow. *Measurement* becomes graph statistics with no ground truth and a composite score; *certification* becomes possible where the truth is known by construction (recall 1.00 in 9/9 trained boolean models — trained networks demonstrably *contain* the wiring of the circuits they compute); *reading* becomes a graph-network problem, and graph networks demonstrably see what feature-only models cannot (0.91–0.96 vs. 0.72–0.76 macro-F1 on depth roles, with transfer to unseen sizes). And the framework’s testability pays off precisely where other approaches hand-wave: we can report, with numbers, that the reading is size-invariant but architecture-specific, and that semantic recovery is not readable from structure by any method.

The method has limitations worth stating plainly. Recovery is functional (input-pair co-wiring), not per-parameter identity: a model could in principle implement the same co-wiring with different gates. Extraction thresholds are per-layer relative; they make the graph scale-free but introduce a free parameter (*rel_thresh*, here 0.5) whose sensitivity we have not exhaustively mapped. The GNN experiments use small, single-task graphs; scaling reading to the circuit graphs of production-scale language models is an open problem — though the extraction itself (blockwise Jacobians) is precisely the kind of computation that scales with tensor cores. And the architecture-specificity of reading

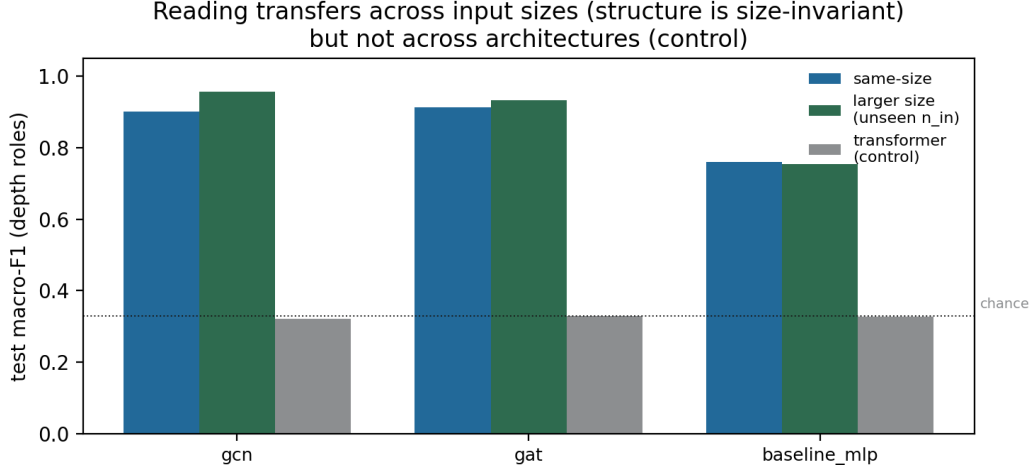


Figure 4: **Reading transfers across sizes, not architectures.** A GNN trained on 7-input MLP circuit graphs reads depth roles on 9-input graphs (never seen) at 0.96 macro-F1, while the feature-only baseline saturates at 0.76. On transformer graphs the same reading collapses to chance for every readout (control).

suggests that *transferable* explicability models will need either richer structural features or explicit cross-architecture training.

None of these limits change the central claim. A model’s computation is a graph; GNOmE extracts it, measures it, and learns to read it — and in doing so converts mechanistic interpretability from a collection of heuristics into a measurable, learnable, testable object of study. The graph is not a metaphor for the computation. It is the computation.

4 Methods

4.1 Models and tasks

MLP. $d_{\text{in}} \rightarrow 32 \rightarrow 32 \rightarrow d_{\text{out}}$ with ReLU, six unit layers (input, linear, ReLU, linear, ReLU, output). **Transformer.** One attention layer plus per-token MLP with residual connections (modular tasks, two one-hot tokens) or two layers over per-bit tokens (boolean tasks); block list is embed $\rightarrow$ (attention, MLP) $^\ell \rightarrow$ readout. All models are trained to 100% train accuracy with Adam. **Boolean tasks.** Random layered DAGs of {AND, OR, XOR} gates over n_{in} bits, n_g gates, fixed seed; the ground-truth graph is the literal generated DAG. **Modular tasks.** $(a + b) \bmod p$ with one-hot inputs; the ground-truth circuit is the canonical Fourier/trigonometric construction [7]: per-frequency cos/sin features, product terms implementing the addition formula, a sum, and an argmax readout.

4.2 Extraction

For each block k with unit dims (d_{k+1}, d_k) , estimate $\mathbf{W}_k[i, j] = \mathbb{E}_x |\partial u_{k+1}[i] / \partial u_k[j]|$: closed-form exact $|\mathbf{W}|$ for linear blocks, diagonal $|f'|$ for elementwise blocks, one autograd pass per output unit otherwise, averaged over batches of 256. Threshold per layer at $\text{rel_thresh} \times$ the layer’s mean nonzero weight; keep edges above the threshold. Residual connections are folded into the block they close. Output: a layered DAG with node ids $u_{k,j}$, layer ranks, and per-edge Jacobian weights.

4.3 Metrics

Sparsity = $1 - |E| / \sum_k d_{k+1} d_k$. **Modularity** via greedy Louvain communities on the undirected graph, clipped to $[0, 1]$. **Effective depth** = longest directed path. **Bottleneck** = $1 - \min_k d_k / \sum_k d_k$ (narrowest layer). **Role diversity** = fraction of distinct (in-degree, out-degree) signatures. **MES** = $0.4 F_1^{\text{rec}} + 0.2 \text{sparsity} + 0.2 \text{modularity} + 0.2 \text{depth fidelity}$ when a ground truth exists (fidelity = $\min(1, d_{\text{ext}}/d_{\text{gt}})$), else the recovery weight redistributes onto the other components. **Recovery** compares input-pair co-wiring: for each pair of inputs, does some hidden unit (extracted) / gate (ground truth) depend on both? Precision, recall, F1 over input pairs.

4.4 Graph networks

GCN layers: $\mathbf{X}' = \text{ReLU}(\mathbf{W}(D^{-1} \mathbf{A} \mathbf{X}))$ with edge weights and self-loops; **GAT** layers: attention $\alpha_{ij} \propto \exp(\text{LeakyReLU}(\mathbf{a}^\top [\mathbf{h}_i; \mathbf{h}_j]))$ weighted by edge strength, two heads. Three stacked layers, hidden width 64, role head on node embeddings. **Node features** (structure-only): normalized out/in degree, mean outgoing/incoming edge weight — no layer index. **Baseline**: MLP on the same features, no edges. Training: Adam, 300 epochs, cross-entropy on depth bins (input/early/mid/late/output); few-shot exp1 uses $K \in \{2, 4, 6\}$ training graphs and held-out graphs; exp2 trains on six 7-input MLP graphs and evaluates on held-out same-size, 9-input unseen-size, and transformer graphs.

4.5 Reproducibility

All numbers are produced by the committed scripts (`benchmarks/run_benchmark.py`, `benchmarks/run_gnn_experiment.py`, `benchmarks/analyze.py`) on CPU (PyTorch 2.10, Python 3.14, 8 threads) and written to committed JSONs (`results/benchmark_cpu.json`, `results/gnn_experiment.json`); figures are regenerated by `benchmarks/make_figures.py`. Full wall time for both experiments: ≈ 8 minutes on CPU. Nothing is simulated or copied from the literature.

References

- [1] Peter W. Battaglia, Jessica B. Hamrick, Victor Bapst, Alvaro Sanchez-Gonzalez, Vinicius Zambaldi, Mateusz Malinowski, Andrea Tacchetti, David Raposo, Adam Santoro, Ryan Faulkner, Caglar Gulcehre, Francis Song, Andrew Ballard, Justin Gilmer, George Dahl, Ashish Vaswani, Kelsey Allen, Charles Nash, Victoria Langston, Chris Dyer, Nicolas Heess, Daan Wierstra, Pushmeet Kohli, Matt Botvinick, Oriol Vinyals, Yujia Li, and Razvan Pascanu. Relational inductive biases, deep learning, and graph networks. *arXiv preprint arXiv:1806.01261*, 2018.
- [2] Arthur Conmy, Adrian-Nicolae Măndoiu, Andrew Battaglia, Neel Nanda, and Stefan Zohren. Towards automated circuit discovery for mechanistic interpretability. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [3] Nelson Elhage, Neel Nanda, Catherine Olsson, Tom Henighan, Nicholas Joseph, Ben Mann, Amanda Askell, Yuntao Bai, Anna Chen, Tom Conerly, Nova DasSarma, Dawn Drain, Deep Ganguli, Zac Hatfield-Dodds, Danny Hernandez, Andy Jones, Jackson Kernion, Liane Lovitt, Kamal Ndousse, Dario Amodei, Tom Brown, Jack Clark, Jared Kaplan, Sam McCandlish, and Chris Olah. A mathematical framework for transformer circuits. *Transformer Circuits Thread*, 2021.
- [4] Javier Ferrando, Gabriele Sarti, Arianna Bisazza, and Marta R. Costa-jussà. A primer on the inner workings of transformer-based language models. *arXiv preprint arXiv:2405.00208*, 2024.

- [5] Sarthak Jain, Byron Voigt, and Byron C. Wallace. Attention is not explanation. In *Proceedings of NAACL-HLT*, 2019.
- [6] Thomas N. Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. In *International Conference on Learning Representations (ICLR)*, 2017.
- [7] Neel Nanda and Tom Lieberum. A mechanistic interpretability analysis of addition. *ARENA blog post*, 2022.
- [8] Neel Nanda, Lawrence Chan, Tom Lieberum, Jess Smith, and Jacob Steinhardt. Progress measures for grokking via mechanistic interpretability. In *International Conference on Learning Representations (ICLR)*, 2023.
- [9] Chris Olah, Nick Cammarata, Ludwig Schubert, Gabriel Goh, Michael Petrov, and Shan Carter. Zoom in: An introduction to circuits. *Distill*, 2020. doi: 10.23915/distill.00024.
- [10] Benjamin Sanchez-Lengeling, Emily Reif, Adam Pearce, and Alexander B. Wiltschko. A gentle introduction to graph neural networks. *Distill*, 2021. doi: 10.23915/distill.00033.
- [11] Mukund Sundararajan, Ankur Taly, and Qiqi Yan. Axiomatic attribution for deep networks. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, 2017.
- [12] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. Graph attention networks. In *International Conference on Learning Representations (ICLR)*, 2018.
- [13] Kevin Wang, Alexandre Variengien, Arthur Conmy, Buck Shlegeris, and Jacob Steinhardt. Interpretability in the wild: a circuit for indirect object identification in GPT-2 small. *arXiv preprint arXiv:2211.00593*, 2022.
- [14] Fred Zhang and Neel Nanda. Towards best practices of activation patching in language models: Metrics and methods. *arXiv preprint arXiv:2309.16042*, 2023.